



## 密码学引论实验报告

姓名: 江李阳, 王健一, 邱泽辉

学号: 202400460042, 202400460037, 202400460046

专业: 密码科学与技术

日期: June 24, 2026

Contents

1 实验目的 2

2 实验环境 2

3 实验原理 2

3.1 Diffie-Hellman 密钥协商协议 . . . . . 2

3.2 认证密钥协商协议 . . . . . 2

3.2.1 端到端密钥协商 . . . . . 3

3.3 传输层安全协议标准 . . . . . 3

4 项目架构 4

4.1 安全目标 . . . . . 4

4.2 流程设计 . . . . . 5

4.3 密码学算法总览 . . . . . 15

5 算法测试效率 15

5.1 正常使用测试 . . . . . 15

5.2 异常测试 . . . . . 15

6 实验结果分析与总结 18

6.1 实验目标达成情况 . . . . . 18

6.2 安全性分析 . . . . . 18

6.3 性能分析 . . . . . 19

6.4 不足与改进方向 . . . . . 19

A 小组分工 22

B AI 使用声明 22

B.1 使用工具信息 . . . . . 22

B.2 生成内容说明 . . . . . 22

C 项目完整代码地址 22

## 1 实验目的

针对 AI 智能体协作航班预定场景, 结合密钥建立协议、电子支付协议等技术, 设计 AI 智能体协作安全协议。本协议的设计参考了 Google 提出的 Agent-to-Agent (A2A) 开源协议规范 [1], 在其基础上增加了基于 PKI 的身份认证、细粒度授权控制和审计回执等密码学安全机制。

## 2 实验环境

- 硬件环境: 13th Gen Intel(R) Core(TM) i5-13420H (12 CPUs), 2.1GHz 处理器; 16384MB RAM 内存。
- 软件环境: Visual studio code
- 操作系统: Arch Linux(kernel:x86\_64 Linux 6.6.87.2-microsoft-standard-WSL2)
- 编程语言: Python 3.14
- 包管理器: uv 0.11.18 (x86\_64-unknown-linux-gnu)

## 3 实验原理

### 3.1 Diffie-Hellman 密钥协商协议

Diffie-Hellman 密钥协商协议基于离散对数问题困难性 (DLP), 实现通信双方  $P_A, P_B$  共享相同会话密钥, 协议基本过程如下:

1.  $P_A$  随机选择  $r_A \in [1, q-1]$ , 计算  $R_A = g^{r_A}$ , 并将  $R_A$  发送给  $P_B$ ;
2.  $P_B$  随机选择  $r_B \in [1, q-1]$ , 计算  $R_B = g^{r_B}$ ,  $K_{AB} = R_A^{r_B}$ , 并将  $R_B$  发送给  $P_A$ ;
3.  $P_A$  计算  $K_{AB} = R_B^{r_A}$ .

以上过程实现了  $P_A, P_B$  共同协商建立了会话密钥  $K_{AB}$ :

$$K_{AB} = R_B^{r_A} = (g^{r_B})^{r_A} = g^{r_B r_A} = (g^{r_A})^{r_B} = R_A^{r_B}$$

但是该密钥协商协议难以抵御中间人攻击, 根本原因在于基础版的 Diffie-Hellman 密钥交换协议缺乏认证, 合法用户无法确认所收到消息来源的真实性。

### 3.2 认证密钥协商协议

认证密钥协商 (authenticated key exchange, AKE) 允许两个或多个用户在不安全网络信道上进行身份和消息认证, 并协商会话密钥, 从而建立安全信道进行通信。

### 3.2.1 端到端密钥协商

端到端密钥协商 (station to station, STS) 是 AKE 协议中的一种, 其本质为一种带有认证功能的 Diffie-Hellman 协议, 使用“认证”的公钥数字签名提供认证性。其认证即认证公钥的真实性, 是通过可信第三方: 证书认证机构 (certification authority, CA) 进行的。用户在 CA 处注册并得到证书 (certificate), 该证书实质是 CA 对用户身份与公钥的数字签名, 其作用是验证用户公钥的合法性, 如 CA 为用户  $P_A$  颁发的证书如下:

$$Cert(P_A) = (ID_A, pk_A, \sigma_{CA,A})$$

其中  $ID_A$  为用户  $P_A$  的唯一标识符,  $pk_A$  为用户  $P_A$  的公钥,  $\sigma_{CA,A}$  为 CA 用自己的私钥  $sk_{CA}$  对消息  $ID_A || pk_A$  的数字签名。STS 协议基本过程如下:

0. 首先通信双方  $P_A, P_B$  需要在 CA 处注册得到各自的证书:

$$Cert(P_A) = (ID_A, pk_A, \sigma_{CA,A}), Cert(P_B) = (ID_B, pk_B, \sigma_{CB,B})$$

1.  $P_A$  随机选择  $r_A \in [1, q-1]$ , 然后计算  $R_A = g^{r_A}$ , 并将  $Cert(P_A), R_A$  发送给  $P_B$ .
2.  $P_B$  随机选择  $r_B \in [1, q-1]$ , 然后计算  $R_B = g^{r_B}$ ,  $K_{AB} = (R_A)^{r_B}$  和  $\sigma_B = Sign_{sk_B}(ID_A || R_B || R_A)$ , 并将  $Cert(P_B), R_B, \sigma_B$  发送给  $P_A$ .
3.  $P_A$  首先利用  $Cert(P_B)$  验证  $pk_B$  的合法性, 若无效则拒绝通信并退出; 使用  $pk_B$  验证签名  $\sigma_B$ , 若无效则拒绝通信并退出; 否则接收, 计算  $K_{AB} = (R_B)^{r_A}$  和  $\sigma_A = Sign_{sk_A}(ID_B || R_A || R_B)$ , 并将  $\sigma_A$  发送给  $P_B$ .
4.  $P_B$  首先使用  $Cert(P_A)$  验证  $pk_A$  的合法性, 若无效则拒绝通信并退出; 使用  $pk_A$  验证签名  $\sigma_A$ , 若无效则拒绝通信并退出; 否则接收, 计算  $K_{AB} = (R_A)^{r_B}$ .

STS 协议包含 Diffie-Hellman 协议的主要步骤, 能够保持对会话密钥的保密性, 并对所发送消息添加了签名认证机制, 因此能有效抵御中间人攻击。

### 3.3 传输层安全协议标准

传输层安全 (transport layer security, TLS) 协议用于在两个通信应用之间提供保密性和认证性, 协议本身由 TLS 握手协议 (TLS handshake) 和 TLS 记录协议 (TLS record) 组成, 其中 TLS 握手协议负责服务器向客户端认证自身进而与客户端协商密钥; TLS 记录协议使用握手协议协商的密钥对消息数据进行加密与认证。TLS1.3 协议采用基于椭圆曲线上的 Diffie-Hellman 密钥协商协议, 其基本过程如下:

1. 客户端随机选择  $r_C = \{0, 1\}^{256}$ ,  $x \in \mathbb{Z}_q$ , 计算  $X = g^x$ , 将  $\{r_C, X\}$  发送给服务器。
2. 服务器端 S 随机选择  $r_S = \{0, 1\}^{256}$ ,  $y \in \mathbb{Z}_q$ , 计算  $Y = g^y$ , S 计算  $Z = X^y$ , 并进一步得到以下密钥

$$\begin{cases} hts_C || hts_S = F_1(Z, r_C || X || r_S || Y), \\ htk_C || htk_S = KDF(hts_C || hts_S, \epsilon) \end{cases}$$

其中  $\epsilon$  为固定标签信息,  $hts_C, hts_S$  分别是用以生成客户端与服务器, 在握手阶段的密钥, 保护握手阶段需要保密的信息。服务器生成签名:

$$\begin{cases} H_1 = H(r_C || \dots || Cert_S) \\ \sigma_S = Sign_{sk_S}(l_{SCV} || H_1) \end{cases}$$

其中  $l_{SCV}$  是固定的标签信息,  $H_1$  表示服务器当前接收与本步将要发送的所有消息杂凑值. 计算:

$$\begin{cases} fk_S = HKDF(hts_S, l_5, \mu) \\ fk_C = HKDF(hts_C, l_5, \mu) \\ H_2 = H(r_C || \dots || Cert_S || \sigma_S) \\ fin_S = HMAC(fk_S, H_2) \end{cases}$$

其中  $l_5$  表示固定的标签信息,  $\mu$  表示 HKDF 的输出长度. 使用  $htk_S$  加密  $Cert_S || \sigma_S || fin_S$ , 将  $\{r_S, Y, Enc_{htk_S}(Cert_S || \sigma_S || fin_S)\}$  发送给客户端.

3. 客户端收到信息后, 计算  $Z = Y^x$ , 同 2 步计算方法, 得到  $hts_C || hts_S$  与  $htk_C || htk_S$ , 使用  $htk_S$  解密信息, 验证  $Cert_S, \sigma_S, fin_S$  是否合法. 计算

$$\begin{cases} H_3 = H(r_C || \dots || Cert_S || Cert_C) \\ \sigma_C = Sign_{sk_C}(l_{CCV} || H_3) \\ H_4 = H(r_C || \dots || Cert_S || \sigma_S || Cert_C || \sigma_C) \\ fin_C = HMAC(fk_C, H_4) \end{cases}$$

其中  $l_{CCV}$  表示固定标签信息. 使用  $htk_C$  加密  $\{Cert_C || \sigma_C || fin_C\}$ , 将密文发送给服务器.

4. 服务器使用  $htk_C$  解密密文, 分别验证  $Cert_C, \sigma_C, fin_C$  是否合法, 计算以下密钥

$$\begin{cases} ats_C || ats_S = F_2(Z, r_C || \dots || fin_S) \\ atk_C || atk_S = KDF(ats_C || ats_S, \epsilon) \\ ems = F_3(Z, r_C || \dots || fin_S) \\ rms = F_3(Z, r_C || \dots || fin_C) \end{cases}$$

其中  $atk_C$  与  $atk_S$  分别表示客户端与服务器的应用流量密钥, 用于记录层的认证加密. 同理客户端也计算得到上述密钥.

## 4 项目架构

### 4.1 安全目标

根据实验指导书所述场景, 可以明确该 AI 智能体协作安全协议应当满足如下安全目标 (包括实验要求和额外添加):

1. 可验证身份绑定: 所有智能体持有 CA 颁发的证书, 接收方通过 CA 公钥验证证书真伪, 从而验证对方身份, 杜绝伪造的智能体接入交互环节.
2. 数据传输机密性: 涉及用户隐私的数据, 包括行程信息、订单价格、授权凭证数据等均需要保密, 杜绝数据泄露风险.
3. 授权不扩散: Alice 生成的授权凭证包括: 总预算、授权权限、授权时效、被授权智能体 ID, 完整内容由 Alice 签名认证. 凭证内容一旦被篡改, 任务随即自动终止, 杜绝授权扩散、越权操作. 同时授权凭证仅单次会话有效, 不可复用.

4. 执行全程可追溯:FlightAgent 完成业务执行后, 必须生成带自身私钥签名的审计回执, 回执包含: 会话 ID、执行时间、订单详情、实际花费、授权凭证摘要、执行智能体 ID。Alice 汇总所有回执后, 可通过对应智能体公钥验证回执真伪, 所有交互日志、凭证、回执可本地留存, 支持事后全流程审计追溯, 不可抵赖。
5. 完整性、时效性与防重放: 协议中所有关键消息都必须能够验证是最新的、只用一次的、未被篡改的, 即效信息内容的完整性校验、防重放、序列号/时间戳验证、会话绑定。

## 4.2 流程设计

根据上述安全要求, 我们设计的 AI 智能体协作航班预定场景协议流程如下:

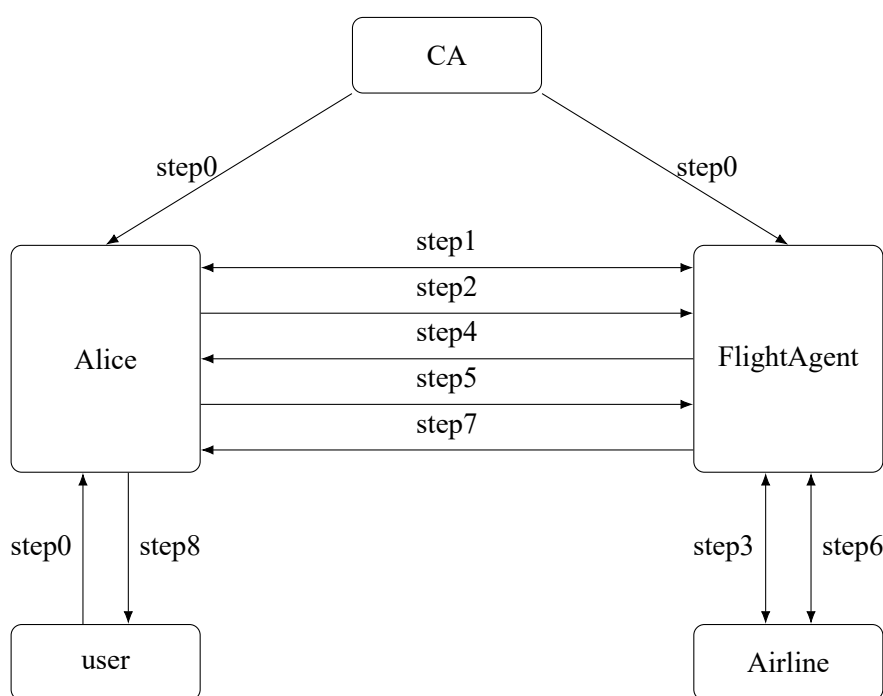


Figure 1: Multi-Agent Flight Booking Protocol

其中 CA 表示证书认证机构,Alice 为用户智能体,FlightAgent 为航班预定智能体,user 为用户,Airline 为航空公司 (也可以是智能体)。上述协议具体步骤如下:

step0 Alice 和 FlightAgent 在 CA 处注册得到各自的证书 certificate,user 向 Alice 发出预定机票的命令:” 预订下周 (2026.06.08-2026.06.16) 青岛—纽约往返机票, 总预算不超过 10000 元, 单人出行”, 此后仅需等待 Alice 与 FlightAgent 协作交互即可, 无需 user 介入操作。

**密码学操作:** CA 使用 Ed25519 为每个 Agent 签发身份证书, 证书绑定 Agent 身份与公钥 (这里 CA 是提前给各方签发证书的, 协议在线运行阶段不涉及与 CA 的交互):

## CA 证书签发—Ed25519

$$(sk_A, pk_A) \leftarrow \text{Ed25519.KeyGen}()$$

$$\text{cert\_body}_A = \{\text{agent\_id}_A, pk_A, \text{permissions}, \text{not\_before}, \text{not\_after}\}$$

$$\sigma_{CA,A} = \text{Sign}(sk_{CA}, \text{canon}(\text{cert\_body}_A)) \quad (\text{Ed25519, 128-bit 安全强度})$$

$$\text{Cert}_A = (\text{cert\_body}_A, \sigma_{CA,A})$$

参数: Ed25519 (Edwards25519 曲线), 证书有效期 2026-06-01 ~ 2026-12-31.

代码: ca/certificate\_authority.py —issue\_agent(), verify\_certificate().

```

1  def issue_agent(self, agent_id, permissions,
2      not_before="2026-06-01T00:00:00+00:00",
3      not_after="2026-12-31T23:59:59+00:00"):
4      # 1. 生成 Agent 的 Ed25519 密钥对
5      signing_private = generate_keypair()
6      signing_public = signing_private.public_key()
7      cert_body = {
8          "agent_id": agent_id,
9          "public_key": b64e(signing_public.public_bytes_raw()),
10         "permissions": permissions,
11         "not_before": not_before, "not_after": not_after,
12         "issuer": "TrustedOfflineCA",
13     }
14     # 2. CA 使用 Ed25519 私钥对证书体签名
15     certificate = {
16         "body": cert_body,
17         "ca_signature": sign(self.private_key, cert_body),
18     }
19     return AgentIdentity(agent_id, permissions, signing_private,
20                           certificate)
21
22 def verify_certificate(self, certificate):
23     body = certificate["body"]
24     # 1. 验证 CA 签名: Verify(pk_CA, cert_body, ca_signature)
25     verify(self.public_key, certificate["ca_signature"], body)
26     # 2. 检查证书有效期窗口
27     current = datetime.now(timezone.utc)
28     if current < datetime.fromisoformat(body["not_before"]):
29         raise ValueError("Certificate not yet valid")
30     if current > datetime.fromisoformat(body["not_after"]):
31         raise ValueError("Certificate has expired")
32     return AgentIdentity(body["agent_id"], body["permissions"],
33                           signing_private=None, certificate=certificate)

```

Listing 1: ca/certificate\_authority.py —issue\_agent() & verify\_certificate()

step1 Alice 与 FlightAgent 之间基于 STS 协议 (认证密钥协商) 进行身份验证与 ECDH 会话密钥协商。

密码学操作:

### Phase 1-2: 认证密钥协商—X25519 ECDH + Ed25519 + HKDF

#### ClientHello (Alice $\rightarrow$ FlightAgent):

$(a, A = g^a) \leftarrow \text{X25519.KeyGen}()$  (临时密钥对)  
 $\text{body}_A = \{\text{session\_id}, \text{agent\_id}_A, A_{\text{pub}}, \text{timestamp}\}$   
 $\sigma_A = \text{Sign}(sk_A, \text{canon}(\text{body}_A))$  (Ed25519)

#### ServerHello (FlightAgent $\rightarrow$ Alice):

$\text{Verify}(pk_{CA}, \text{cert\_body}_A, \sigma_{CA,A}) \rightarrow pk_A$  (CA 证书链验证)  
 $\text{Verify}(pk_A, \text{canon}(\text{body}_A), \sigma_A)$  (身份签名验证)  
 $(b, B = g^b) \leftarrow \text{X25519.KeyGen}()$   
 $\text{shared\_secret} = B^a = A^b = g^{ab}$  (X25519 ECDH)  
 $\text{HKDF-SHA256}(\text{salt} = \text{session\_id}, \text{info} = \text{"a2a-secure..."}, L = 96)$   
 $\rightarrow k_{\text{enc}}[0 : 32] \parallel k_{\text{mac}}[32 : 64] \parallel k_{\text{log}}[64 : 96]$

参数: X25519 (Curve25519), HKDF-SHA256, 派生 96 字节密钥材料, 三把密钥用途分离 (加密/完整性/审计).

代码: protocol/handshake.py —HandshakeInitiator, HandshakeResponder; crypto/kdf.py —derive\_keys().

```

1  # --- Alice 侧: make_client_hello() ---
2  def make_client_hello(self):
3      session_id = "sess-" + uuid.uuid4().hex
4      ephemeral_priv = generate_private() # X25519 临时密钥对
5      body = {
6          "session_id": session_id,
7          "agent_id": self.identity.agent_id,
8          "x25519_public": public_to_b64(ephemeral_priv.public_key()),
9          "timestamp": now_iso(),
10     }
11     hello = {
12         "phase": "client_hello", "body": body,
13         "certificate": self.identity.certificate,
14         "signature": sign(self.identity.signing_private, body), #
15         Ed25519
16     }
17     self.pending[session_id] = ephemeral_priv
18     return session_id, {SECURE_HANDSHAKE_URI: hello}
19
20 # --- FlightAgent 侧: process_client_hello() ---
21 def process_client_hello(self, hello):

```



```

21 body, cert = hello["body"], hello["certificate"]
22 # Step 1: 验证 Alice 的 CA 证书
23 alice_identity = self.ca.verify_certificate(cert)
24 # Step 2: 验证 Alice 对 hello body 的 Ed25519 签名
25 verify(alice_identity.public_key, hello["signature"], body)
26 # Step 3: 身份绑定 — agent_id 必须与证书一致
27 if body["agent_id"] != cert["body"]["agent_id"]:
28     raise IdentityError("Identity binding mismatch")
29 # Step 4: X25519 ECDH + HKDF 密钥派生
30 my_ephemeral = generate_private()
31 shared_secret = exchange(my_ephemeral,
32                           public_from_b64(body["x25519_public"]))
33 session_keys = derive_keys(shared_secret, body["session_id"])
34 # kenc[0:32] || kmac[32:64] || klog[64:96] (HKDF-SHA256, L=96)
35 server_body = {
36     "session_id": body["session_id"],
37     "agent_id": self.identity.agent_id,
38     "x25519_public": public_to_b64(my_ephemeral.public_key()),
39     "timestamp": now_iso(),
40 }
41 server_hello = {
42     "phase": "server_hello", "body": server_body,
43     "certificate": self.identity.certificate,
44     "signature": sign(self.identity.signing_private, server_body)
45 ,
46 }
47 return body["session_id"], session_keys, server_hello

```

Listing 2: protocol/handshake.py —ClientHello (Alice) &amp; ServerHello (FlightAgent)

step2 Alice 生成授权凭证 AuthToken, 包括预算、行程、授权信息、授权权限、授权时效、被授权智能体 ID 等, 对这些信息计算杂凑值并签名, 将得到的 AuthorizationCredential 发送给 FlightAgent。

密码学操作:

### Phase 3: 授权凭证—Ed25519 签名

```

AuthToken = {issuer, delegatee, session_id, budget_cny,
              permissions, nonce, validity, itinerary}
 $\sigma_{Auth} = \text{Sign}(sk_A, \text{canon}(\text{AuthToken}))$  (Ed25519)
Passport = {AuthToken,  $\sigma_{Auth}$ , CertA}

```

参数: budget\_cny=10000, permissions=["flight.search", "flight.book"], valid\_minutes=10, nonce=token\_hex(16).

代码: protocol/auth\_token.py —make\_authorization(), make\_passport().

```

1 def make_authorization(issuer, delegatee_agent_id, session_id,

```

```

2         budget_cny=10000, permissions=None,
3         itinerary=None, valid_minutes=10):
4     if permissions is None:
5         permissions = ["flight.search", "flight.book"]
6     auth_payload = {
7         "issuer_agent_id":    issuer.agent_id,
8         "delegatee_agent_id": delegatee_agent_id,
9         "session_id":        session_id,
10        "budget_cny":         budget_cny,
11        "permissions":        permissions,
12        "valid_from":         now_iso(),
13        "valid_until":        (datetime.now(timezone.utc)
14                               + timedelta(minutes=valid_minutes)).isoformat
15    ),
16        "nonce":              secrets.token_hex(16), # 一次性 nonce
17        "nonce_used":         False,
18        "itinerary":          itinerary or {},
19    }
20    # Alice 对完整授权负载进行 Ed25519 签名
21    return {
22        "authorization": auth_payload,
23        "signature":      sign(issuer.signing_private, auth_payload),
24    }
25
26 def make_passport(issuer, delegatee_agent_id, session_id,
27                  budget_cny=10000, permissions=None, itinerary=None):
28     auth = make_authorization(issuer, delegatee_agent_id, session_id,
29                               budget_cny, permissions, itinerary)
30     # 打包: AuthToken + 签名 + Alice 的证书
31     return {
32         "clientId":         issuer.agent_id,
33         "sessionId":        session_id,
34         "state":             {"authorization": auth["authorization"]},
35         "certificate":       issuer.certificate,
36         "signature":         auth["signature"],
37     }

```

Listing 3: protocol/auth\_token.py —make\_authorization() &amp; make\_passport()

step3 FlightAgent 接收到授权后先验证签名, 如果签名不通过则拒绝服务, 直接退出; 否则, 根据授权凭证的信息执行即机票查询操作, 将查询结果, 包括具体航班、日期、实际价格, 时间戳等信息一并计算杂凑值并签名, 将签名和加密后的查询信息发给 Alice 以询问是否购买预定。

密码学操作 (多层验证链):

## Phase 3: 安全信封解密 + 授权验证 + 查询

## FlightAgent 接收侧:

ReplayWindow.check(session\_id, seq, timestamp) (防重放,  $\pm 30s$  窗口)  
 HMAC<sub>verify</sub>( $k_{mac}$ , envelope\_without\_hmac, tag) (HMAC-SHA256 完整性)  
 AES-256-GCM.Decrypt( $k_{enc}$ , nonce, ct, AAD)  $\rightarrow$  plaintext (AEAD 解密)  
 Verify( $pk_{CA}$ , cert\_body<sub>A</sub>,  $\sigma_{CA,A}$ )  $\rightarrow pk_A$  (CA 证书链)  
 Verify( $pk_A$ , canon(AuthToken),  $\sigma_{Auth}$ ) (授权签名验证)  
 检查: delegatee\_id  $\stackrel{?}{=}$  agent:flight, session\_id  $\stackrel{?}{=}$  当前会话,  
 budget  $\leq 10000$ , permissions  $\supseteq \{\text{search, book}\}$

## 查询结果加密返回:

results = BookingService.search\_flights(origin, dest, date)  
 $\sigma_{Result} = \text{Sign}(sk_F, \text{canon}(\{\text{flights, timestamp, session\_id}\}))$   
 response = AES-256-GCM.Encrypt( $k_{enc}$ , results  $\parallel \sigma_{Result}$ )

参数: AES-256-GCM (nonce=12B, key=32B), HMAC-SHA256 (key=32B), Replay-Window (skew=30s).

代码: protocol/envelope.py — open\_envelope(); agents/flight\_agent.py — process\_secure\_request(), \_handle\_query().

```

1  # --- protocol/envelope.py: open_envelope() ---
2  def open_envelope(envelope, session_keys, replay_window):
3      keys = session_keys
4      # Step 1: 防重放 (seq + timestamp,  $\pm 30s$  窗口)
5      replay_window.accept(envelope["session_id"],
6                           envelope["seq"], envelope["timestamp"])
7      # Step 2: HMAC-SHA256 完整性校验 (排除 hmac 字段)
8      mac_input = canon({k: v for k, v in envelope.items() if k != "hmac"})
9      if not verify_b64(keys["kmac"], mac_input, envelope["hmac"]):
10         raise IntegrityError("HMAC integrity check failed")
11     # Step 3: AES-256-GCM AEAD 解密
12     plaintext = decrypt(b64d(envelope["nonce"]),
13                        b64d(envelope["ciphertext"]),
14                        keys["kenc"], canon(envelope["aad"]))
15     return plaintext
16
17  # --- agents/flight_agent.py: _verify_authorization() ---
18  def _verify_authorization(self, passport, session_id):
19     auth = passport["state"]["authorization"]
20     # CA 证书链验证: Verify(pk_CA, cert_body, ca_signature)
21     alice_identity = self.ca.verify_certificate(passport["certificate"])
22     # Alice 的 Ed25519 授权签名
23     verify_sig(alice_identity.public_key, passport["signature"], auth)
  
```

```

24     # 约束检查
25     if auth["delegatee_agent_id"] != self.identity.agent_id:
26         raise AuthorizationError("Wrong delegatee")
27     if auth["session_id"] != session_id or auth.get("nonce_used"):
28         raise AuthorizationError("Invalid session or reused nonce")
29     if auth["budget_cny"] > 10000:
30         raise AuthorizationError("Budget exceeds limit")
31     if not {"flight.search", "flight.book"}.issubset(auth["permissions"]):
32         raise AuthorizationError("Missing required permissions")
33     return auth
34
35     # --- agents/flight_agent.py: _handle_query() ---
36     def _handle_query(self, auth, klog):
37         flights = self.booking_service.search_flights(
38             origin=auth["itinerary"].get("from", "Qingdao"),
39             destination=auth["itinerary"].get("to", "New York"),
40             depart_date=auth["itinerary"].get("depart_date", "2026-06-08"),
41             passengers=auth["itinerary"].get("passengers", 1),
42         )
43         result_body = {
44             "session_id": auth["session_id"], "flights": flights,
45             "searched_at": now_iso(),
46             "executor_agent_id": self.identity.agent_id,
47         }
48         # FlightAgent 对查询结果签名:  $\sigma_F = \text{Sign}(sk_F, result\_body)$ 
49         return {
50             "type": "flight_query_result", "body": result_body,
51             "signature": do_sign(self.identity.signing_private,
52                                 result_body),
53             "certificate": self.identity.certificate,
54         }

```

Listing 4: protocol/envelope.py —open\_envelope() ; agents/flight\_agent.py —\_verify\_authorization() & \_handle\_query()

step4 Alice 收到 FlightAgent 发送来的信息后先验证签名, 如果签名不通过则拒绝服务并直接退出; 否则, 根据查询结果, 判断是否购买预定, 发送 BookRequest 给 FlightAgent 允许其预定机票。

密码学操作:

#### Phase 4: 查询结果验证 + 预订确认

$\text{AES-256-GCM.Decrypt}(k_{enc}, \text{response}) \rightarrow \text{results} \parallel \sigma_{Result}$

$\text{Verify}(pk_{CA}, \text{Cert}_F) \rightarrow pk_F$ ,  $\text{Verify}(pk_F, \text{canon}(\text{results}), \sigma_{Result})$  (查询结果验签)

$\text{BookRequest} = \{\text{flight\_no}, \text{confirm}\}$ ,  $\text{Enc}_{k_{enc}}(\text{BookRequest}) \rightarrow \text{FlightAgent}$

代码: agents/alice\_agent.py —query\_flights(), confirm\_booking().

```

1  # --- query_flights(): Phase 3 — 发送加密查询, 验证签名结果 ---
2  async def query_flights(self, client, session_id,
3                          itinerary=None, budget_cny=10000, ...):
4      payload = self._build_request_payload(
5          action="query", session_id=session_id,
6          budget_cny=budget_cny, itinerary=itinerary)
7      keys = self._session_keys[session_id]
8      # AES-256-GCM + HMAC 安全信封封装
9      metadata = seal_envelope(payload, session_id, keys,
10                               seq=self._next_seq(session_id))
11     response = await _send_once(client, "flight query request",
12                                  metadata)
13     # 校验 FlightAgent 返回结果的 Ed25519 签名
14     if response.get("type") == "flight_query_result":
15         flight_id = self.ca.verify_certificate(response["certificate"])
16         verify(flight_id.public_key,
17                response["signature"], response["body"])
18     return response
19
20 # --- confirm_booking(): Phase 4-5 — 发送加密预订请求 ---
21 async def confirm_booking(self, client, session_id,
22                            flight_no, itinerary=None, budget_cny=10000):
23     payload = self._build_request_payload(
24         action="confirm", session_id=session_id,
25         budget_cny=budget_cny, itinerary=itinerary,
26         flight_no=flight_no)
27     keys = self._session_keys[session_id]
28     metadata = seal_envelope(payload, session_id, keys,
29                               seq=self._next_seq(session_id))
30     return await _send_once(client, "booking confirmation", metadata)

```

Listing 5: agents/alice\_agent.py —query\_flights() & confirm\_booking()

step5 FlightAgent 收到 Alice 发送来的 BookRequest 信息, 同样验证签名, 不通过则拒绝服务并退出, 否则执行 step6。

**密码学操作:** 重复 step3 的安全信封解密与 HMAC 验证流程 (同 open\_envelope()), 验签通过后执行预订。

```

1  # process_secure_request() 中 action=="confirm" 分支
2  # 1. open_envelope(): 防重放 + HMAC 完整性 + AES-256-GCM 解密 (同 step3)
3  plaintext = open_envelope(envelope, keys, replay)
4  request   = json.loads(plaintext)
5  # 2. _verify_authorization(): CA 证书链 + 授权签名 + 约束检查 (同 step3)
6  passport = request["metadata"][SECURE_PASSPORT_URI]
7  auth     = self._verify_authorization(passport, session_id)
8  # 3. 分发至 _handle_confirm()
9  flight_no = request.get("flight_no", "CA981")
10 return self._handle_confirm(auth, flight_no, keys["klog"])

```

Listing 6: agents/flight\_agent.py —process\_secure\_request() confirm 分支

step6 FlightAgent 在航空公司 Airline 处预定机票, 将具体订单 ID、航空公司信息、机票信息、具体花费等信息作为收据 Receipt, 签名并发回给 Alice。

密码学操作:

#### Phase 5-6: 订单执行 + 签名审计回执

```

order = BookingService.create_order(flight_no, passengers, route)
receipt_body = {session_id, executed_at, order, actual_spend, auth_digest, executor_id}
auth_digest = SHA-256(canon(AuthToken))    (授权摘要绑定)
 $\sigma_{Receipt}$  = Sign( $sk_F$ , canon(receipt_body))    (Ed25519 不可抵赖)
Klog_HMAC = HMAC( $k_{log}$ , canon(receipt_body))    (审计链绑定)

```

参数: Ed25519 签名, SHA-256 杂凑, HMAC-SHA256 (key= $k_{log}$ , 32B).

代码: protocol/receipt.py —make\_receipt(); agents/booking\_service.py —create\_order().

```

1  # --- protocol/receipt.py: make_receipt() ---
2  def make_receipt(executor, session_id, order, authorization, klog):
3      receipt_body = {
4          "session_id":      session_id,
5          "executed_at":    now_iso(),
6          "order":          order,
7          "actual_spend_cny": order["price_cny"],
8          # 绑定至授权: auth_digest = SHA-256(canon(AuthToken))
9          "authorization_digest": sha256_text(authorization),
10         "executor_agent_id": executor.agent_id,
11     }
12     return {
13         "type":      "signed_audit_receipt",
14         "body":      receipt_body,
15         # 不可抵赖:  $\sigma_F = \text{Sign}(sk_F, receipt\_body)$  (Ed25519)
16         "signature": sign(executor.signing_private, receipt_body),
17         "certificate": executor.certificate,
18         # 审计链绑定:  $Klog\_HMAC = HMAC(klog, receipt\_body)$ 
19         "klog_hmac": compute_b64(klog, canon(receipt_body)),
20     }
21
22  # --- agents/flight_agent.py: _handle_confirm() ---
23  def _handle_confirm(self, auth, flight_no, klog):
24      itinerary = auth.get("itinerary", {})
25      order = self.booking_service.create_order(
26          flight_no=flight_no,
27          passengers=itinerary.get("passengers", 1),
28          route=f"{itinerary.get('from', 'Qingdao')}-"

```

```

29         f"{itinerary.get('to','New York')} round trip",
30         depart_date=itinerary.get("depart_date","2026-06-08"),
31         return_date=itinerary.get("return_date","2026-06-16"),
32     )
33     if order["price_cny"] > auth["budget_cny"]:
34         raise AuthorizationError("Price exceeds authorized budget")
35     return make_receipt(self.identity, auth["session_id"],
36                        order, auth, klog)

```

Listing 7: protocol/receipt.py —make\_receipt(); agents/flight\_agent.py —\_handle\_confirm()

step7 Alice 验证签名, 保存审计记录, 并将结果报告给 user。

密码学操作:

#### Phase 6: Alice 侧回执验证—双重验证

$\text{Verify}(pk_{CA}, \text{Cert}_F) \rightarrow pk_F$  (CA 证书链验证)

$\text{Verify}(pk_F, \text{canon}(\text{receipt\_body}), \sigma_{\text{Receipt}})$  (Ed25519 回执验签)

$\text{HMAC}_{\text{verify}}(k_{\text{log}}, \text{canon}(\text{receipt\_body}), \text{Klog\_HMAC})$  (审计链完整性)

通过后写入审计日志:

```
AuditLogger.log(event_type="receipt", detail_digest=SHA-256(receipt),
klog_hmac).
```

代码: agents/alice\_agent.py—verify\_receipt(); audit/logger.py—AuditLogger;  
audit/store.py—AuditStore.

```

1  # --- agents/alice_agent.py: verify_receipt() ---
2  def verify_receipt(self, receipt, session_id):
3      keys = self._session_keys.get(session_id)
4      if keys is None:
5          raise ValueError(f"Unknown session: {session_id}")
6      # 1. 验证 FlightAgent 的 CA 证书
7      flight_identity = self.ca.verify_certificate(receipt["certificate"]
8      ])
9      # 2. Ed25519 回执签名 + Klog HMAC 验证 (委托至协议层)
10     return verify_receipt_protocol(receipt,
11                                   flight_identity.public_key,
12                                   keys["klog"])
13
14 # --- protocol/receipt.py: verify_receipt() ---
15 def verify_receipt(receipt, executor_public_key, klog):
16     body = receipt["body"]
17     # 不可抵赖: Verify(pk_F, receipt_body, sigma_F)
18     verify(executor_public_key, receipt["signature"], body)
19     # 审计链: HMAC(klog, receipt_body) 必须匹配 klog_hmac
20     expected = compute_b64(klog, canon(body))
21     if receipt.get("klog_hmac") != expected:

```



```

21         raise ValueError("Klog HMAC mismatch - audit chain broken")
22     return True

```

Listing 8: agents/alice\_agent.py —verify\_receipt(); protocol/receipt.py —verify\_receipt()

step8 Alice 将最终结果报告给用户, 包括成功预定或中途发生的异常信息。

### 4.3 密码学算法总览

协议使用的密码学原语及参数汇总:

| 用途    | 算法                     | 参数                          | 安全强度          |
|-------|------------------------|-----------------------------|---------------|
| 身份签名  | Ed25519 (Edwards25519) | 256-bit 密钥, 512-bit 签名      | 128-bit       |
| 密钥协商  | X25519 ECDH            | 256-bit 密钥, Curve25519      | 128-bit       |
| 密钥派生  | HKDF-SHA256            | salt=session_id, L=96B      | 128-bit       |
| 数据加密  | AES-256-GCM            | 256-bit 密钥, 96-bit nonce    | 256-bit       |
| 完整性校验 | HMAC-SHA256            | 256-bit 密钥                  | 256-bit       |
| 消息摘要  | SHA-256                | 256-bit 输出                  | 128-bit (抗碰撞) |
| 防重放   | ReplayWindow           | ±30s 时钟偏移窗口, (sid,seq) 去重   | —             |
| 审计绑定  | Klog HMAC-SHA256       | 密钥 $k_{log}$ (32B), 绑定回执与日志 | 256-bit       |

Table 1: 协议密码学算法总览

## 5 算法测试效率

### 5.1 正常使用测试

首先启动 FlightAgent, 然后启动 Alice, 可以看到 Alice 可以与 FlightAgent 正确协商密钥并进行合作订购机票, 效果如下图: 整体耗时 23.41ms, 其中各个阶段耗时由上可得下表:

Table 2: 正常流程各阶段性能数据

| 阶段        | 描述                                 | 耗时        | 占比    |
|-----------|------------------------------------|-----------|-------|
| Step 0    | 获取 FlightAgent AgentCard (证书获取与连接) | 117.00 ms | 99.5% |
| Phase 1-2 | 身份验证 + ECDH 密钥协商 (X25519+HKDF)     | 17.47 ms  | 74.6% |
| Phase 3   | 加密航班查询 (AES-GCM 封装/解封)             | 3.44 ms   | 14.7% |
| Phase 4-5 | 确认预订 (签名回执生成)                      | 2.49 ms   | 10.6% |
| Phase 6   | 审计回执验证 (Ed25519 验签 +HMAC)          | 0.25 ms   | 1.1%  |
| 总计        | —                                  | 23.41 ms  | —     |

### 5.2 异常测试

- 重放攻击测试: 截取历史合法消息, 重复发送, 结果如下图:



```

[mio@LYJ A2A]$ uv run python main.py
Using CPython 3.14.6 interpreter at: /usr/sbin/python3.14
Removed virtual environment at: .venv
Creating virtual environment at: .venv
Installed 39 packages in 36ms
[FlightAgent] Starting A2A server on http://localhost:10001
...
[FlightAgent] AgentCard extensions:
- https://example.edu/a2a/extensions/secure-handshake/v1
- https://example.edu/a2a/extensions/secure-envelope/v1
- https://example.edu/a2a/extensions/secure-flight-passpor
t/v1
INFO: Started server process [598]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:10001 (Press CTR
L+C to quit)
INFO: 127.0.0.1:55498 - "GET /.well-known/agent-card.js
on HTTP/1.1" 200 OK
INFO: 127.0.0.1:55498 - "POST / HTTP/1.1" 200 OK
INFO: 127.0.0.1:55498 - "POST / HTTP/1.1" 200 OK
INFO: 127.0.0.1:55498 - "POST / HTTP/1.1" 200 OK

[mio@LYJ A2A]$ uv run python main.py --client
=====
Alice AI 智能体 - 航班预订客户端 (自动模式)
=====

- Step 0: 获取 FlightAgent AgentCard -
[V] 已连接到 FlightAgent, 耗时 0.117s

- Phase 1-2: 身份认证 + ECDH 密钥协商 -
[V] 握手完成, session_id=sess-cb136d6642104787a3b4667dcf3abf58
密码学: X25519 ECDH + Ed25519 签名 + HKDF-SHA256
派生密钥: kenc(32B) || kmac(32B) || klog(32B)
耗时: 17.47ms

- Phase 3: 加密航班查询 -
[Alice] Enc_kenc(QueryRequest || AuthToken)
[V] 查询完成, 耗时 3.45ms

可用航班 (2 个):
CA981: Qingdao>New York 出发 2026-06-08 08:30 抵达 2026-06-08 22:
00 ¥9288 (3座)
MU587: Qingdao>New York 出发 2026-06-08 11:00 抵达 2026-06-09 01:
30 ¥8500 (1座)

- Phase 4-5: 确认预订 CA981 -
[Alice] Enc_kenc(ConfirmRequest || flight_no)
[V] 预订完成, 耗时 2.49ms

- Phase 6: 验证审计回执 -
订单号: ORD-803B5241
航班: CA981
路线: Qingdao-New York round trip
价格: ¥9288 CNY
授权摘要: b51a288a97bd13a4...
[Alice] Verify(pk_Flight, receipt_body, σ_Flight)
[Alice] Verify_HMAC(klog, receipt_body)
[V] 回执验证通过 - 审计链完整, 耗时 0.23ms

总耗时: 23.41ms
=====
[mio@LYJ A2A]$

```

Figure 2: 正常使用测试

```

[mio@LYJ A2A]$ ls
agents audit ca crypto introduction main.py pro
tocol __pycache__ pyproject.toml README.md server
tests trust_store.json uv.lock
[mio@LYJ A2A]$ uv run python main.py
[FlightAgent] Starting A2A server on http://localhost
:10001 ...
[FlightAgent] AgentCard extensions:
- https://example.edu/a2a/extensions/secure-handsha
ke/v1
- https://example.edu/a2a/extensions/secure-envelop
e/v1
- https://example.edu/a2a/extensions/secure-flight-
passport/v1
INFO: Started server process [2716]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:10001 (Pr
ess CTRL+C to quit)
INFO: 127.0.0.1:46563 - "GET /.well-known/agent-c
ard.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:46563 - "POST / HTTP/1.1" 200 OK
INFO: 127.0.0.1:46563 - "POST / HTTP/1.1" 200 OK
INFO: 127.0.0.1:46563 - "POST / HTTP/1.1" 200 OK

C:\Windows\system32\cmd.exe
[mio@LYJ ~]$ cd A2A/
[mio@LYJ A2A]$ uv run tests/manual/01_replay_attack.py
=== 攻击测试 1: 重放攻击 (Replay Attack) ===

[V] 拦截成功 - 重放攻击被阻止
原理: ReplayWindow 维护 (session_id, seq) 已见集合, 相同 seq 重复发送时拒绝
(时间窗口 ±30s)
耗时: 2.77ms
[mio@LYJ A2A]$

```

Figure 3: 重放攻击测试

- 授权扩权测试: 手动篡改授权凭证预算金额, 验证签名校验是否拦截篡改请求, 禁止扩权 (这里将预算手动改为 50000, 权限手动添加一个 "admin.delete\_all" 字段):

```
[mio@LYJ A2A]$ uv run python tests/manual/02_escalation.py
=== 攻击测试 2: 授权扩权 (Authorization Escalation) ===

[√] 预算扩权拦截成功 (10000 → 50000 CNY)
    原理: Ed25519 签名锁定 AuthToken 全部字段, 篡改 budget 后 canonical(AuthToken) 变化 → Verify() 失败
    耗时: 4.36ms

[√] 权限扩权拦截成功 (添加 admin.delete_all)
    原理: 同上 - permissions 也是签名输入, 篡改后签名失效
    耗时: 4.59ms
[mio@LYJ A2A]$ |
```

Figure 4: 授权扩权测试

- 消息篡改测试: 修改传输中的订单价格、行程信息, 验证 HMAC 完整性校验是否识别篡改:

```
[mio@LYJ A2A]$ uv run python tests/manual/03_tampering.py
=== 攻击测试 3: 消息篡改 (Message Tampering) ===

[√] 密文篡改拦截成功
    原理: AES-256-GCM AEAD - 解密时认证标签校验失败 (InvalidTag)
    耗时: 3.30ms

[√] 信封篡改拦截成功
    原理: HMAC-SHA256 覆盖整个信封, seq 被改后 compute_hmac(envelope) ≠ envelope.hmac → 拒绝
    耗时: 3.44ms
[mio@LYJ A2A]$ |
```

Figure 5: 消息篡改测试

- 伪造身份测试: 使用非法凭证、伪造智能体身份 (这里使用非法第三方 CA 签发的证书) 发起请求, 验证身份绑定机制是否拦截非法接入。

```
=== 攻击测试 4: 伪造身份 (Identity Forgery) ===

[√] 拦截成功 - 伪造身份被拒绝
    原理: CA 证书链验证 - FlightAgent 用可信 CA 公钥验证证书签名, 非法 CA 签发的证书 σ_CA ≠ σ_CA → 验签失败
    耗时: 4.82ms
[mio@LYJ A2A]$ |
```

Figure 6: 伪造身份测试

综上该协议能正确检测出这些异常情况并及时响应, 确保系统的安全性。测试异常检测数据总结如下表:

Table 3: 异常攻击检测结果汇总

| 攻击类型    | 检测结果 | 防护机制                                                                  | 耗时      |
|---------|------|-----------------------------------------------------------------------|---------|
| 重放攻击    | 拦截成功 | ReplayWindow 维护 (session_id, seq) 已见集合, 相同 seq 重复发送时拒绝                | 2.77 ms |
| 预算扩权    | 拦截成功 | Ed25519 签名锁定 AuthToken 全部字段, 篡改 budget 后 canonical() 变化导致 Verify() 失败 | 4.36 ms |
| 权限扩权    | 拦截成功 | 同上——permissions 也是签名输入, 篡改后签名失效                                       | 4.59 ms |
| 密文篡改    | 拦截成功 | AES-256-GCM AEAD 解密时认证标签校验失败 (Invalid-Tag)                            | 3.30 ms |
| 信封/序号篡改 | 拦截成功 | HMAC-SHA256 覆盖整个信封, seq 被改后 compute_kmac $\neq$ envelope.hmac, 拒绝     | 3.44 ms |
| 伪造身份    | 拦截成功 | CA 证书链验证: 非法 CA 签发的证书 $\sigma'_{CA} \neq \sigma_{CA}$ , 验签失败          | 4.82 ms |

## 6 实验结果分析与总结

### 6.1 实验目标达成情况

本次实验围绕”Agent-to-Agent (A2A) 安全通信协议”的设计与实现展开, 完整覆盖了从证书颁发、密钥协商、授权管理到安全通信与审计回执的全流程。实验主要达成了以下目标:

- PKI 信任基础设施搭建:** 实现了基于 Ed25519 的 CA 证书颁发与验证机制, 支持 Agent 身份的公钥绑定与链式信任传递。
- 前向保密密钥交换:** 采用 X25519 ECDH + HKDF-SHA256 方案完成会话密钥的双方一致派生, 派生出加密密钥  $k_{enc}$ 、完整性密钥  $k_{mac}$  与审计密钥  $k_{log}$ , 实现三钥用途分离。
- 细粒度授权控制:** 通过带数字签名的 AuthToken + Passport 机制, 将预算上限、权限列表、有效期等约束条件以密码学方式绑定, 防止越权操作。
- 多层安全防护体系:** 在传输层同时部署 AES-256-GCM (机密性 + 认证)、HMAC-SHA256 (完整性)、ReplayWindow (抗重放) 和 Ed25519 签名 (不可否认性), 构成纵深防御。
- 可审计回执机制:** 每笔订单生成包含签名与 HMAC 审计链的回执, Alice 可独立验证 FlightAgent 的执行行为, 实现事后可追溯。

### 6.2 安全性分析

通过对六类典型攻击场景的测试, 协议在各层面均表现出有效的防护能力:

Table 4: 安全防护能力汇总

| 安全属性    | 攻击场景    | 防护机制                                                   |
|---------|---------|--------------------------------------------------------|
| 机密性     | 密文篡改/窃听 | AES-256-GCM AEAD 加密, 篡改后解密认证标签校验失败                     |
| 完整性     | 消息/信封篡改 | HMAC-SHA256 覆盖全信封, 任何字段变更均导致 MAC 不匹配                   |
| 抗重放     | 重放攻击    | ReplayWindow 维护 (session_id, seq) 已见集合, 超时窗口 $\pm 30s$ |
| 认证与身份绑定 | 伪造身份/冒充 | Ed25519 CA 证书链验证, 非法证书签名不匹配即拒绝                         |
| 授权约束    | 预算/权限扩权 | AuthToken 全部字段经 Ed25519 签名锁定, 篡改导致验签失败                 |
| 不可否认性   | 订单抵赖    | 回执含 Ed25519 签名 $\sigma_F$ + HMAC 审计链, 双方均可验证           |

所有异常测试均在 5 ms 内完成检测并拒绝恶意请求, 表明协议的安全开销可控且响应及时。

### 6.3 性能分析

正常流程总耗时为 23.41 ms, 各阶段性能特征如下:

- **Step 0 (证书获取, 117 ms):** 占总耗时的主要部分。该阶段涉及网络 I/O (HTTP 请求获取 AgentCard), 属于一次性握手开销, 后续复用 session 无需重复。
- **Phase 1–2 (ECDH+HKDF, 17.47 ms):** X25519 标量乘法与 HKDF 派生是计算密集型操作, 但仅需在会话建立时执行一次。
- **Phase 3–6 (业务操作, 共 6.18 ms):** 查询、预订、验证等业务操作的密码学开销均在毫秒级, AES-GCM 加解密与 Ed25519 签名/验签效率较高。
- **异常检测 (2.77–4.82 ms):** 各类攻击的检测延迟均匀分布在 3–5 ms 区间, 不会成为性能瓶颈。

总体而言, 除去网络 I/O 的证书获取步骤后, 核心密码学协议的总耗时仅为 23.41 ms, 满足实时交互的性能要求。

### 6.4 不足与改进方向

尽管实验基本达到了设计目标, 但当前实现仍有以下局限:

1. **单点 CA 依赖:** 当前仅使用一个 CA, 若 CA 私钥泄露则整个信任体系崩溃。改进方案可采用多 CA 联盟或基于门限签名的分布式 CA。
2. **无前向安全性 (PFS) 保障:** 长期密钥泄露可能导致历史会话密钥被推导。可引入 ratchet 机制 (如 Signal 协议的双棘轮算法) 实现完美前向保密。
3. **ReplayWindow 为内存存储:** 重启后丢失状态。改进方案可将已见 (session\_id, seq) 对持久化至数据库或使用滑动窗口日志。

4. **缺少密钥撤销机制:** Agent 私钥泄露后无法即时吊销证书。可增加 CRL（证书吊销列表）或 OCSP 在线状态查询。
5. **测试覆盖面有限:** 当前仅覆盖了 6 类手动构造的异常场景, 未进行模糊测试 (fuzzing) 或形式化验证。

## References

- [1] A2A Project. Agent-to-agent (a2a) protocol. <https://github.com/a2aproject/A2A>, 2024. Accessed: 2026-06-24.

## A 小组分工

Table 5: 实验小组成员分工

| 姓名  | 学号           | 负责内容                 |
|-----|--------------|----------------------|
| 江李阳 | 202400460042 | 协议分析设计, 代码撰写, 实验报告撰写 |
| 王健一 | 202400460037 | 审计实验报告, 修改整合与润色实验报告  |
| 邱泽辉 | 202400460046 | 协议分析设计与调研, 代码撰写      |

## B AI 使用声明

本报告在编写过程中使用了人工智能辅助工具, 具体使用情况声明如下:

### B.1 使用工具信息

- 工具名称: WorkBuddy Code (基于大语言模型的 AI 编程助手)
- 用途范围:
  - 实验报告  $\text{\LaTeX}$  排版调试、语法纠错与格式优化
  - 代码片段的提取、整理与嵌入文档 (`\lstlisting` 格式化)
  - 测试截图数据的人工核对与表格填充.

### B.2 生成内容说明

1. **文本内容:** 实验原理描述、协议步骤分析、结果分析章节由作者独立撰写, AI 辅助进行语句润色与格式调整。
2. **图表数据:** 所有测试截图、性能数据均来自真实运行环境的人工操作记录, AI 未参与任何数据生成或伪造。

## C 项目完整代码地址

<https://github.com/hxhdhcjmet/a2aProtocal.git>